

CS714: Secure Computation

Adithya Vadapalli

IIT Kanpur

January 5, 2026

Who Am I?

- ① Adithya Vadapalli, Assistant Professor from August 2023 at IIT Kanpur.
- ② Office: KD317
- ③ Research Interests: Applied Cryptography

Who are You?

- 1 Quick Survey, # B.Tech, # M.Tech, # PhD students?
- 2 Quick Survey, How many of you have taken CS670?
- 3 Your expectations from this course?

Book

Secure Multiparty Computation and Secret Sharing *by Ronald Cramer, Ivan Damgard, and Jesper Nielsen*

Course Website

<https://github.com/privacy-iitk/cs714-26-Winter/blob/main/README.md>

CS670 v CS714

CS670 was a lot about intuitions. CS714 is much more rigorous.

- ① HelloIITK will be used to make announcements. We will send group e-mails.
- ② Course website will have the slides and any additional reading materials.
- ③ Gradescope for viewing exams.
- ④ Always welcome to my office to ask questions.
- ⑤ Exam will be traditional closed book.

Prerequisites

- ① Slightly mathematically heavy: Need mathematical maturity.
- ② This is a theoretical course.
- ③ There will not be any programming.
- ④ There will be quite a few proofs.

We will be using Acadly to take attendance!

Grading Mechanism

- Attendance (via Acadly): 2%
- Midsem: 35%
- Endsem: 35%
- Quiz 1: 10%
- Quiz 2: 10%
- Assignments ($\times 4$): 8%

In a nutshell, please be honest!

Auctions

- ① There is an auctioneer, A , who is selling an item.
- ② There are bidders, $P_1, \dots, P_n$.
- ③ Each bidder P_i has a private value M_i , the maximum value they are willing to pay for the item.

Need for Privacy

What can an auctioneer do if they learn a bidder's private value M_i .

Benchmarking

- 1 Suppose you run a company. You are interested in how your company runs compared to others.
- 2 Some parameters you may be interested in are:
 - profits relative to size
 - salaries
 - productivity
- 3 The companies would like to keep these values private, though

Datamining

- ① Public insitutions maintain databases about citizens, like heath, demography, etc.
- ② Statistics from such databases could be very useful.
- ③ However, that entails privacy issues.

A Possible Solution

Trusted Party Solution

- 1 Suppose there exists a trusted party T , that everyone trusts.
- 2 Every party can give their private inputs to T , who does the computation.

Problem with a Trusted Party

- 1 There is a single point of failure.
- 2 Everyone needs to trust one party (but, these parties were not willing to trust each other).

A *slightly* formal definition

- ① We have n players, $P_1, P_2, \dots, P_n$.
- ② They hold secret inputs $x_1, x_2, \dots, x_n$.
- ③ Their goal is to compute: $y \leftarrow f(x_1, x_2, \dots, x_n)$, with two properties:
 - Correctness: Correct value of y is computed.
 - Privacy: y is the *only* new information that is released.

An Informal Definition

A party, P , holds a secret x . The goal is to share the secret among multiple parties such that, no individual parties (except the party, P) know the secret, but a collection of parties can recover the secret!

Additive Secret Shares

A party, P , chooses r_1 , r_2 , and r_3 from $\mathbb{Z}_p$, such that $r_1 + r_2 + r_3 = x$.

Two Main Properties

Privacy

No single party should be able to learn anything about the secret.

Correctness

If a *certain number* of parties combine with each other, then they can recover the secret.

Secret Sharing

A Secret Sharing Mechanism

- 1 P_1 wants to secret share s , P_1 chooses a random numbers $r_1 \in \mathbb{Z}_p$ and $r_2 \in \mathbb{Z}_p$.
- 2 P_1 computes: $r_3 = s - r_1 - r_2$.
- 3 P_1 privately sends (r_1, r_3) to P_2 , (r_1, r_2) to P_3 .

Privacy

- If some hacker, hacks into the computer of P_2 or P_3 , he **will not** learn the secret.
- If some hacker, hacks into the computer of *both* P_2 or P_3 , he **will** learn the secret.

Correctness

If any two parties combine, they can compute: x .

A More Formal Argument for Privacy

- Consider P_2 : He knows (r_1, r_3) , but not r_2 .
- Now consider some random $x_0 \in \mathbb{Z}_p$.
- Could x be equal to x_0 from P_2 's point-of-view?
- If x was equal to x_0 , then: $r_2 = x_0 - r_1 - r_3$.

From P_2 's point-of-view, can x be any value $\in \mathbb{Z}_p$!

The Problem

- 1 We have three parties, P_0 , P_1 , and P_2 .
- 2 They hold secret inputs x_0 , x_1 , and x_2 respectively.
- 3 Their goal is to obtain $v = x_0 + x_1 + x_2$.
- 4 Secrecy needs to be maintained.

The Solution

- 1 P_i samples $r_i^{(1)}, r_i^{(2)}$ at random and sets $r_i^{(3)} \leftarrow x_i - r_i^{(1)} + r_i^{(2)}$. P_i sends $r_i^{(1)}$ to P_1 , $r_i^{(2)}$ to P_2 , and $r_i^{(3)}$ to P_3 . (of course this involves sending one of the shares to itself!)
- 2 Each P_j computes: $s^{(\ell)} \leftarrow r_1^{(\ell)} + r_2^{(\ell)} + r_3^{(\ell)}$, for $\ell \neq j$. They announce, $s^{(\ell)}$ to everyone.
- 3 All parties compute: $v \leftarrow s^{(1)} + s^{(2)} + s^{(3)} \pmod p$

Correctness

Why is the above protocol correct?

Correctness Analysis

This is straightforward.

$$v = \sum_j s_j \bmod p = \sum_j \sum_i r_{i,j} \bmod p = \sum_i \sum_j r_{i,j} \bmod p = \sum_i x_i \bmod p.$$

Privacy

Why does this protocol give no new information to the parties?

Privacy Analysis

- 1 In the first step, x_1, x_2, x_3 are secret shared. This is private. Why?
- 2 Then, s_1, s_2, s_3 are announced. Why does this not give any new information?
- 3 The protocol allows you to learn v . Focus on P_1 . It knows s_2 and s_3 . The only new information it learn is s_1 . But, it can compute $s_1 = v - s_2 - s_3$.

Secure Multiplication

The Problem

- 1 P_1 's private input is $a \in \mathbb{Z}_p$. P_2 's private input is $b \in \mathbb{Z}_p$. P_3 has no input.
- 2 They would like to compute $a \cdot b$

The Protocol

- 1 P_1 distributes a_1, a_2, a_3 ; the shares of a . P_2 distributes b_1, b_2, b_3 ; the shares of b .
- 2 P_1 locally computes: $u_1 = a_2b_2 + a_2b_3 + a_3b_2$, P_2 locally computes: $u_2 = a_3b_3 + a_1b_3 + a_3b_1$, P_3 computes: $u_3 = a_1b_1 + a_1b_2 + a_2b_1$
- 3 Finally, they use Secure Addition Protocol to compute $u_1 + u_2 + u_3$.

Secure Multiplication

The Protocol

- ① P_1 distributes a_1, a_2, a_3 ; the shares of a . P_2 distributes b_1, b_2, b_3 ; the shares of b .
- ② P_1 locally computes: $u_1 = a_2b_2 + a_2b_3 + a_3b_2$, P_2 locally computes: $u_2 = a_3b_3 + a_1b_3 + a_3b_1$, P_3 computes: $u_3 = a_1b_1 + a_1b_2 + a_2b_1$
- ③ Finally, they use Secure Addition Protocol to compute $u_1 + u_2 + u_3$.

The Third Party's Role

What happens if the third reveals to P_1 and P_2 what it sees?

Secure Multiplication

The Protocol

- ① P_1 distributes a_1, a_2, a_3 ; the shares of a . P_2 distributes b_1, b_2, b_3 ; the shares of b .
- ② P_1 locally computes: $u_1 = a_2b_2 + a_2b_3 + a_3b_2$, P_2 locally computes: $u_2 = a_3b_3 + a_1b_3 + a_3b_1$, P_3 computes: $u_3 = a_1b_1 + a_1b_2 + a_2b_1$
- ③ Finally, they use Secure Addition Protocol to compute $u_1 + u_2 + u_3$.

Can we make it simpler?

Rather running the secure addition protocol, can the parties just announce, u_1 , u_2 , and u_3 ?

Matchmaking using Secure Multiplication

- 1 Suppose Alice is interested in knowing if Bob wants to go out with her.
- 2 Bob is asking himself, if there is any interest from Alice's side.
- 3 Can they find out if there is mutual interest, while keeping their preferences private?

What if players do not follow instructions?

Malicious Players

What are the ways in which players can be malicious?

Malicious Players

- ① changing the inputs
- ② not following the protocol

What if players do not follow instructions?

Changing Inputs

How can Alice (for example), take advantage in the matchmaking example by not following instructions?

Solutions

Unfortunately, there are no good solutions to this problem.

What if players do not follow instructions?

Malicious Players in Secure Addition

Consider the secure addition protocol: In what ways can the parties deviate from the protocol?

Deviating from the protocol in Secure Addition

- ① it could do the secret sharing in wrong way!
- ② it could announce the results incorrectly

Solutions

We would like add mechanisms in the protocol that *detect* deviation from the protocol.

What if players do not follow instructions?

Incorrect Sharing

- 1 P_1 could pick $x_1 \neq \tilde{r}_1^{(1)} + \tilde{r}_2^{(1)} + \tilde{r}_3^{(1)}$ (is this a problem?)
- 2 P_1 could send $r_1^{(1)}, r_1^{(3)}$ to P_2 and send $\tilde{r}_1^{(1)}, r_1^{(2)}$ to P_3 (why is this more egregious?)

Announcing the wrong sum

Suppose that P_1 deviates from the protocol and announces $s'_2 \neq s_2$

Solutions

We would like add mechanisms in the protocol that *detect* deviation from the protocol.

Models of Secure Computation

Semi-honest setting

All the players follow the protocol. Also called passive security.

Malicious/General Security

Some of the players deviate from the protocol!

Shamir Secret Sharing: Example Setup

Parameters

- Field: $\mathbb{Z}_{11}$
- Number of parties: $n = 5$
- Threshold: $t = 2$

Goal

Share a secret s such that any $t + 1 = 3$ shares reconstruct s , and any $t = 2$ shares reveal no information.

Polynomial for Sharing

Shamir Construction

Choose a random polynomial of degree t :

$$f(x) = s + a_1x + a_2x^2 \pmod{11}$$

Concrete Choice

$$s = 6, \quad a_1 = 4, \quad a_2 = 7$$

$$f(x) = 6 + 4x + 7x^2 \pmod{11}$$

Computing the Shares

Share Definition

Each party P_i receives

$$s_i = f(i), \quad i = 1, \dots, 5$$

Evaluations

$$s_1 = f(1) = 6$$

$$s_2 = f(2) = 9$$

$$s_3 = f(3) = 4$$

$$s_4 = f(4) = 2$$

$$s_5 = f(5) = 3$$

$$(s_1, s_2, s_3, s_4, s_5) = (6, 9, 4, 2, 3)$$

Reconstruction Setting

Available Shares

Assume parties P_3, P_4, P_5 cooperate:

$(3, 4), (4, 2), (5, 3)$

Method

Reconstruct $f(x)$ using Lagrange interpolation over $\mathbb{Z}_{11}$.

Lagrange Basis Polynomials

General Formula

$$\ell_j(x) = \prod_{\substack{k=3 \\ k \neq j}}^5 \frac{x - k}{j - k}$$

Computed Bases

$$\ell_3(x) = (x - 4)(x - 5)$$

$$\ell_4(x) = -(x - 3)(x - 5)$$

$$\ell_5(x) = 6(x - 3)(x - 4)$$

Interpolating the Polynomial

Reconstruction

$$\begin{aligned} f(x) &= 4 \ell_3(x) + 2 \ell_4(x) + 3 \ell_5(x) \\ &\equiv 7x^2 + 4x + 6 \pmod{11} \end{aligned}$$

Recovered Polynomial

$$f(x) = 6 + 4x + 7x^2$$

Recovering the Secret

Secret Extraction

The secret is the constant term:

$$s = f(0)$$

Result

$$s = 6$$

Why Fewer Shares Do Not Work

Security Argument

With only $t = 2$ shares:

- Infinitely many degree-2 polynomials fit
- Each yields a different secret

Conclusion

$t + 1$ shares are necessary and sufficient to reconstruct s .

That's all for today!